

Banking Management System - MySQL Capstone Project

Project Objective

Design and implement a Banking Management System using MySQL to manage branches, customers, bank accounts, and transactions using normalization, CRUD, joins, subqueries, views and transactions

Problem Statement

Banks need to securely maintain customer accounts and transaction records while ensuring data consistency and accuracy.

MySQL Workflow

- Requirement Analysis
- Identify Entities
- ER Diagram
- Normalization (1NF → 3NF)
- Create Database
- Create Tables
- Insert Sample Data
- CRUD Operations
- Numeric, String & Date Functions
- GROUP BY & HAVING
- Joins & Subqueries
- Views
- Transactions

Database Design

- Branches(branch_id PK, branch_name, location)
- Customers(customer_id PK, customer_name, phone, email, address)
- Account_Types(account_type_id PK, account_type_name)
- Accounts(account_id PK, branch_id FK, customer_id FK, account_type_id FK, account_number, balance, opening_date, status)
- Transactions(transaction_id PK, account_id FK, transaction_type, amount, transaction_date, description)

Insert Data

```
INSERT INTO Branches VALUES (1,'MG Road','Bengaluru','IFSC001'),(2,'Banjara Hills','Hyderabad','IFSC002'),(3,'Governorpet','Vijayawada','IFSC003'),(4,'Dwaraka Nagar','Vizag','IFSC004'),(5,'Brodipet','Guntur','IFSC005');
```

INSERT INTO Customers VALUES

(1,'Aarav','9000000001','aarav@mail.com','Bengaluru'),(2,'Bhavya','9000000002','bhavya@mail.com','Hyderabad'),(3,'Charan','9000000003','charan@mail.com','Vijayawada'),(4,'Deepika','9000000004','deepika@mail.com','Vizag'),(5,'Esha','9000000005','esha@mail.com','Guntur'),(6,'Farhan','9000000006','farhan@mail.com','Nellore'),(7,'Gopi','9000000007','gopi@mail.com','Kurnool'),(8,'Hari','9000000008','hari@mail.com','Ongole'),(9,'Isha','9000000009','isha@mail.com','Warangal'),(10,'John','9000000010','john@mail.com','Chennai');

INSERT INTO Account_Types VALUES (1,'Savings'),(2,'Current');

INSERT INTO Accounts VALUES

(101,1,1,1,'100000001',25000,'2025-01-01','Active'),
(102,2,2,1,'100000002',50000,'2025-01-02','Active'),
(103,2,2,2,'200000003',120000,'2025-01-03','Active'),
(104,3,3,1,'100000004',35000,'2025-01-04','Active'),
(105,4,4,1,'100000005',65000,'2025-01-05','Active'),
(106,4,4,2,'200000006',150000,'2025-01-06','Active'),
(107,5,5,2,'200000007',90000,'2025-01-07','Active'),
(108,6,1,1,'100000008',18000,'2025-01-08','Inactive'),
(109,7,2,1,'100000009',27000,'2025-01-09','Active'),
(110,8,3,2,'200000010',78000,'2025-01-10','Active'),
(111,9,4,1,'100000011',45000,'2025-01-11','Active'),
(112,9,4,2,'200000012',99000,'2025-01-12','Active'),
(113,10,5,1,'100000013',60000,'2025-01-13','Active'),
(114,1,1,2,'200000014',85000,'2025-01-14','Active'),
(115,3,3,2,'200000015',110000,'2025-01-15','Active');

INSERT INTO Transactions VALUES

(1001,101,'Deposit',1000,'2025-07-01','Sample Deposit'),
(1002,102,'Withdrawal',6000,'2025-07-02','Sample Withdrawal'),
(1003,103,'Transfer',11000,'2025-07-03','Sample Transfer'),
(1004,104,'Deposit',16000,'2025-07-04','Sample Deposit'),
(1005,105,'Withdrawal',21000,'2025-07-05','Sample Withdrawal'),
(1006,106,'Transfer',26000,'2025-07-06','Sample Transfer'),
(1007,107,'Deposit',31000,'2025-07-07','Sample Deposit'),
(1008,108,'Withdrawal',36000,'2025-07-08','Sample Withdrawal'),
(1009,109,'Transfer',41000,'2025-07-09','Sample Transfer'),
(1010,110,'Deposit',46000,'2025-07-10','Sample Deposit'),
(1011,111,'Withdrawal',1000,'2025-07-11','Sample Withdrawal'),
(1012,112,'Transfer',6000,'2025-07-12','Sample Transfer'),
(1013,113,'Deposit',11000,'2025-07-13','Sample Deposit'),

(1014,114,'Withdrawal',16000,'2025-07-14','Sample Withdrawal'),
(1015,115,'Transfer',21000,'2025-07-15','Sample Transfer'),
(1016,101,'Deposit',26000,'2025-07-16','Sample Deposit'),
(1017,102,'Withdrawal',31000,'2025-07-17','Sample Withdrawal'),
(1018,103,'Transfer',36000,'2025-07-18','Sample Transfer'),
(1019,104,'Deposit',41000,'2025-07-19','Sample Deposit'),
(1020,105,'Withdrawal',46000,'2025-07-20','Sample Withdrawal'),
(1021,106,'Transfer',1000,'2025-07-21','Sample Transfer'),
(1022,107,'Deposit',6000,'2025-07-22','Sample Deposit'),
(1023,108,'Withdrawal',11000,'2025-07-23','Sample Withdrawal'),
(1024,109,'Transfer',16000,'2025-07-24','Sample Transfer'),
(1025,110,'Deposit',21000,'2025-07-25','Sample Deposit'),
(1026,111,'Withdrawal',26000,'2025-07-26','Sample Withdrawal'),
(1027,112,'Transfer',31000,'2025-07-27','Sample Transfer'),
(1028,113,'Deposit',36000,'2025-07-28','Sample Deposit'),
(1029,114,'Withdrawal',41000,'2025-07-29','Sample Withdrawal'),
(1030,115,'Transfer',46000,'2025-07-30','Sample Transfer');

Business Reports

1. List all customer details.
2. Display all active bank accounts.
3. Change the status of an account from **Active** to **Inactive**.
4. Display customer names along with their account numbers and account types.
5. Display account number, branch name, and branch city for all accounts
6. Find customers with the highest account balance.
7. Concatenate customer name with account number. Eg.: Aarav - ACC1001
8. Display the ceiling and floor value of every transaction amount.
9. Find the absolute difference between the opening balance and current balance.
10. Display all customers and the branches where they hold accounts.
11. Calculate total deposits by branch.
12. Find the total balance available across all accounts.
13. Find the highest, lowest, and average account balance.
14. Calculate the total amount deposited and the total amount withdrawn.
15. Show branch-wise customer count.
16. Count Savings and Current accounts.
17. Find customers who own more than one account.
18. Find the latest transaction for each account.
19. Find the customer(s) having the highest account balance.
20. Display accounts whose balance is greater than the average account balance.
21. Show monthly transaction totals with the month name and rounded total amount.
22. Find customers who have never made any transaction.
23. Display inactive accounts.

24. List the top 5 accounts by balance.
25. Count transactions by type.
26. Generate monthly transaction summaries.
27. Display account balances branch-wise.
28. Display the customer name in uppercase along with the rounded account balance.
29. Create a view to display all active account details along with customer and branch information.
30. Write a transaction to transfer **₹10,000** from one account to another using **START TRANSACTION**, **COMMIT**, and **ROLLBACK** to ensure data consistency.
31. Create a stored procedure to display all accounts belonging to a particular customer.
 - Input: customer_id
 - Display account number, account type, balance, and account status.
32. Create a stored procedure to display all transactions for a particular account.
 - Input: account_id
 - Display transaction type, amount, transaction date, and description.
33. Create a trigger on the Transactions table.
 - Whenever a new transaction is inserted, automatically store the transaction ID, account ID, transaction type, amount, and transaction date in a Transaction_Log table.
34. Create a trigger on the Accounts table.
 - If an account's balance is updated to a negative value, prevent the update and display an appropriate error message.